

Tipologie di esercizi:

- Induzione strutturale
- Scoping statico e dinamico
 - * Descrivere la catena statica e la CRT
 - * Completamento del codice
- Scoping e binding
- Passaggio di parametri
 - * Per valore
 - * Per riferimento
- Ricorsione e ricorsione in coda
- Semantica di un'istruzione

Induzione strutturale

1. Alberi binari: $BTree$: $\begin{cases} foglio \in BTree \\ T_1, T_2 \in BTree \rightarrow branch(T_1, T_2) \in BTree \end{cases}$

$$l : BTree \rightarrow \mathbb{N} \text{ n° foglie} : \begin{cases} l(foglio) = 1 \\ l(branch(T_1, T_2)) = l(T_1) + l(T_2) \end{cases}$$

$$h : BTree \rightarrow \text{altezza} : \begin{cases} h(foglio) = 0 \\ h(branch(T_1, T_2)) = \max(h(T_1), h(T_2)) + 1 \end{cases}$$

Dimostrare $\forall T \in BTree, l(T) \leq 2^{h(T)}$ (Tesi)

Base

$T = foglio$

$$l(T) = 1 \quad h(T) = 0 \rightarrow l(T) \leq 2^{h(T)}$$

$$1 \leq 2^0$$

$$1 \leq 1 \Rightarrow l(T) \leq 2^{h(T)}$$

Passo Induttivo

Hp. Induttiva $T_1, T_2 \in BTree$ tali che $l(T_1) \leq 2^{h(T_1)}$ e $l(T_2) \leq 2^{h(T_2)}$

Mostriamo che anche per T vale la tesi:

$$T = branch(T_1, T_2) \Rightarrow l(T) = 2^{h(T)}$$

$$l(T) = l(branch(T_1, T_2))$$

$$\stackrel{\text{def } l}{=} l(T_1) + l(T_2)$$

$$\leq 2^{h(T_1)} + 2^{h(T_2)}$$

hp ind

$$h(T) = h(\text{branch}(T_1, T_2))$$

$$\stackrel{\text{def } h}{=} \max(h(T_1), h(T_2)) + 1$$

Prendiamo $h = \max(h(T_1), h(T_2)) \Rightarrow h \geq h(T_1) \wedge h \geq h(T_2)$

$$l(T) = l(T_1) + l(T_2)$$

$$\leq 2^{h(T_1)} + 2^{h(T_2)}$$

$$\leq 2^h + 2^h$$

$$= 2(2^h)$$

$$= 2^{h+1}$$

$$= 2^{\max(h(T_1), h(T_2)) + 1}$$

$$= 2^{h(T)}$$

Quindi: $l(T) \leq 2^{h(T)}$

Scoping statico e dinamico

```
1 {  
    int a = 0;  
  
    (*)  
  
    while (a ≤ 1) {  
        int x; ← locale al while  
  
        (**)  
  
        x = fun();  
        a++;  
    }  
}
```

Inserire codice in (*) e (**) che faccia produrre alla chiamata di `x = fun()` valori uguali in caso di scoping statico e valori diversi in caso di scoping dinamico.

- Per essere eseguibile bisogna definire la funzione `fun()` in (*)
- `fun()` deve avere un ambiente non locale perchè le regole di scoping abbiano effetto sul risultato
- il riferimento non locale in `fun()` deve essere dichiarato sia nel contesto di definizione di `fun()` che nel contesto di esecuzione di `fun()`

Modificare `a` è rischioso perchè è nella condizione nel `while`, quindi scegliamo di modificare `x`.
`x` non è inizializzata alla definizione quando `fun()` è chiamata la prima volta.

```
(*)  
int x = 2;  
int fun() {  
    return x;  
}
```

```
(**)  
  
x = a;
```

Il codice finale è:

```
{  
    int a = 0;  
  
    int x = 2;  
    int fun() {  
        return x;  
    }  
  
    while (a ≤ 1) {  
        int x;  
  
        x = a;  
  
        x = fun();  
        a++;  
    }  
}
```

Con scoping statico la chiamata di `fun()` restituisce la `x` globale che non viene mai modificata, quindi in entrambe le chiamate la `x` restituita vale 2.

Con scoping dinamico la chiamata di `fun()` restituisce la `x` nell'ambiente della chiamata, quindi la `x` nel `while` che alla prima iterazione vale `x = a = 0` (globale non modificato prima della chiamata). Successivamente `a++` incrementa a 1 la variabile globale `a` e alla seconda chiamata di `fun()` restituisce la `x` interna al `while` che ora vale 1.

2 (Catena statica e CRT)

```

{
  int x = 1;
  int y = 2;

  void a() {
    int z = x + y;
  }

  void b() {
    int x = 2;
    int w = 4;

    void c() {
      int x = y + w;
      a();
    }

    y = x + y;
    c();
  }

  b();
}

```

Catena di annidamento:

```

main (x, y)
├─ a (z)
└─ b (x, w)
    └─ c (x)

```

$$Sd(main) = 0$$

$$Sd(a) = Sd(b) = 1$$

$$Sd(c) = 2$$

Sequenza di chiamate

main $\rightarrow$ b $\rightarrow$ c $\rightarrow$ a

Scoping statico

Mostrare come evolve lo stack spiegando come si calcolano i link statici e come si risale all'rda corretto che contiene la variabile di riferimento.

link dinamico

link statico

Numero di volte che bisogna risalire la catena statica (sequenza di link statici) per trovare il genitore statico

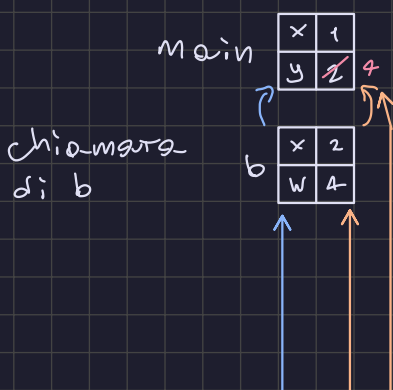
$$K = Sd(chiamante) - Sd(chiamato) + 1$$

$$= Sd(main) - Sd(b) + 1$$

$$= 0 - 1 + 1$$

$$= 0$$

$$\Rightarrow CS(b) = main$$



esecuzione
di b

$$y = x + y$$

Formula da usare:

$$N_b = Sd(uso) - Sd(def)$$

Dice di quanto risalire la catena statica per trovare l'RdA del blocco che definisce a

Il blocco in cui viene usato a

Il blocco più vicino che definisce a nella catena di annidamento

x è locale, quindi non bisogna calcolare nulla
y non locale, quindi calcoliamo N_y

$$N_y = Sd(b) - Sd(main) = 1$$

$$y = \underset{2}{x} + \underset{2}{y} = 4$$

c

x 8

Link statico: $K_c = Sd(b) - Sd(c) + 1$

$$= 1 - 2 + 1 = 0$$

$$\Rightarrow CS(c) = b$$

Esecuzione: $x = \underset{4}{y} + \underset{4}{w} = 8$
locale N_y N_w

$$\left. \begin{array}{l} N_y = Sd(c) - Sd(main) = 2 \\ N_w = Sd(c) - Sd(b) = 1 \end{array} \right\} \text{Risolto della CS}$$

Chiamata
di a

a

z 5

Link statico: $K_a = Sd(c) - Sd(a) + 1$

$$= 2 - 1 + 1 = 2$$

$$\Rightarrow CS(a) = CS(CS(c)) = CS(b) = main$$

Esecuzione di a: $z = \underset{1}{x} + \underset{4}{y}$
locale N_x N_y

$$N_x = Sd(a) - Sd(main) = 1$$

$$= N_y = 1$$

Dopo la chiamata tutte le chiamate terminano e lo stack viene deallocato

Scoping dinamico

Definire la CRT e come viene gestita

main

x	→ main, 1
y	→ main, 2
w	
z	

main → b

x	→ b, 2 → main, 1
y	→ main, 2 ← 4
w	→ b, 4
z	

Esecuzione di: b

$$y = x + y$$

b → c

Esecuzione di: c

x	→ c, ? → b, 2 → main, 1
y	→ main, 4
w	→ b, 4
z	

$$x = y + w$$

c → o

Esecuzione di: o

x	→ c, 8 → b, 2 → main, 1
y	→ main, 4
w	→ b, 4
z	→ z, 12 ←

$$z = x + y$$